

Ch.Sanhith Reddy
2303A52251
Assignment -13

Task:01

Refactoring Data Transformation

Logic

Scenario

You are maintaining a data preprocessing script where numerical transformations are written using verbose loops.

Task Description

- Review the legacy code that computes transformed values using an explicit loop.
- Use an AI tool to suggest a more Pythonic refactoring approach.
- Refactor the code using list comprehensions or helper functions while preserving the output.

Legacy Code

```
values = [2, 4, 6, 8, 10]
```

```
doubled = []
```

```
for v in values:
```

```
    doubled.append(v * 2)
```

```
print(doubled)
```

Expected Output

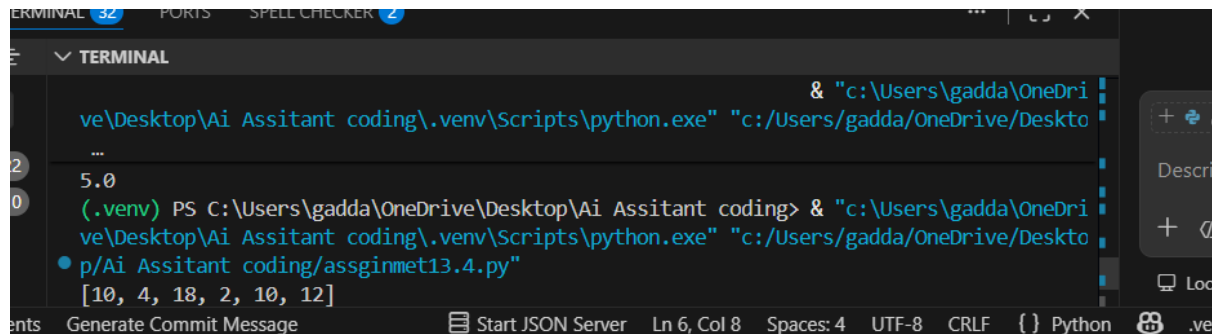
```
[4, 8, 12, 16, 20]
```

Prompt: for this code remove the unwanted lines for code refactoring

Code:

```
ssginmet13.4.py 7 ...
#task 1
values=[5, 2, 9, 1, 5, 6]
s=[value*2 for value in values]
print(s)
```

Output:



```
... & "c:\Users\gadda\OneDrive\Desktop\Ai Assitant coding\venv\Scripts\python.exe" "c:/Users/gadda/OneDrive/Desktop/p/Ai Assitant coding/assginmet13.4.py"
5.0
(.venv) PS C:\Users\gadda\OneDrive\Desktop\Ai Assitant coding> & "c:\Users\gadda\OneDrive\Desktop\Ai Assitant coding\venv\Scripts\python.exe" "c:/Users/gadda/OneDrive/Desktop/p/Ai Assitant coding/assginmet13.4.py"
[10, 4, 18, 2, 10, 12]
```

Observation:

The legacy code uses a **for loop with append()** to manually double each element in the list.

This approach works correctly but is **verbose and less Pythonic** for simple transformations.

It can be refactored using a **list comprehension** to make the code shorter, cleaner, and more readable while producing the same output [4, 8, 12, 16, 20]

Task:02

Improving Text Processing Code

Readability

Scenario

You are working on a log message generator that builds messages word by word.

Task Description

- Analyze the legacy code that constructs a sentence using repeated string concatenation.

- Ask AI to suggest a more efficient and readable approach.
- Refactor the code accordingly while keeping the final output unchanged.

Legacy Code

```
words = ["Refactoring", "with", "AI", "improves",
"quality"]
```

```
message = ""
```

```
for w in words:
```

```
    message += w + " "
```

```
print(message.strip())
```

Expected Output

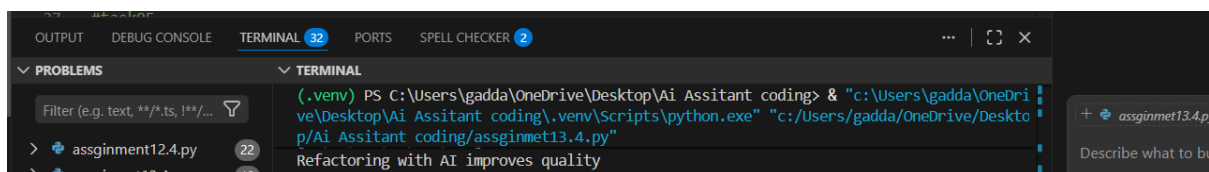
Refactoring with AI improves quality

Prompt:for this code make it the code readable by applying the code refactoring

Code:

```
5
6 #task02
7 words = ["Refactoring", "with", "AI", "improves", "quality"]
8 concat= " ".join(words)
9 print(concat)
```

Output:



Observation:

1. The legacy code builds the sentence using **repeated string concatenation inside a loop**.
2. This approach is **less efficient and less readable**, especially for longer lists of words.

3. It can be refactored using "`.join(words)`", which is more Pythonic, efficient, and produces the same output: *"Refactoring with AI improves quality"*.

Task:03

Safer Access to Configuration Data

Scenario

You are maintaining a configuration loader where dictionary keys may or may not exist depending on deployment settings.

Task Description

- Review the legacy code that manually checks for dictionary keys.
- Use AI suggestions to refactor the code using safer dictionary access methods.
- Ensure the behavior remains the same for missing keys.

Legacy Code

```
config = {"host": "localhost", "port": 8080}
```

```
if "timeout" in config:
```

```
    print(config["timeout"])
```

```
else:
```

```
    print("Default timeout used")
```

Expected Output

Default timeout used

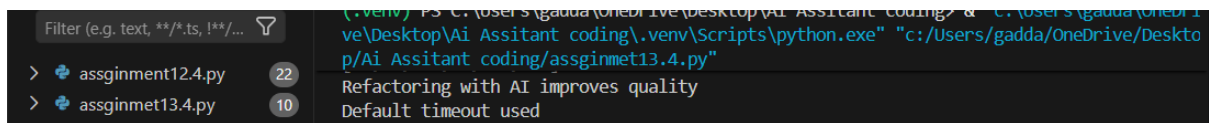
Prompt:

For this legacy code remove unwanted lines of code give me code readable by code refactor

Code:

```
#task03
config = {"host": "localhost", "port": 8080}
print(config.get("timeout", "Default timeout used"))
#task05 - Refactored utility module using missing
```

Output:



```
(.venv) PS C:\Users\gadga\OneDrive\Desktop\Ai Assitant coding> & "C:\Users\gadga\OneDrive\Desktop\Ai Assitant coding\.venv\Scripts\python.exe" "c:/Users/gadga/OneDrive/Desktop/Ai Assitant coding/assginmet13.4.py"
Refactoring with AI improves quality
Default timeout used
```

Observation

The legacy code **manually checks if the key exists in the dictionary** using if "timeout" in config.

This approach works but makes the code **longer and less concise** for simple key access.

it can be refactored using **config.get("timeout", "Default timeout used")** for safer and more Pythonic dictionary access while keeping the same behavior.

Task:04

Scenario

You are enhancing a utility module that performs different operations based on user input.

Task Description

- Examine the multiple if–elif conditions used to determine operations.
- Ask AI to suggest a cleaner, scalable alternative.
- Refactor the logic using mapping techniques while preserving functionality.

Legacy Code

```
action = "divide"
```

```
x, y = 10, 2
```

```
if action == "add":
```

```
result = x + y
```

```
elif action == "subtract":
```

```
result = x - y
```

```
elif action == "multiply":
```

```
result = x * y
```

```
elif action == "divide":
```

```
result = x / y
```

```
else:
```

```
result = None
```

```
print(result)
```

Expected Output

5.0

Prompt:for this code perform the division without performing any operation

Code:

```
import operator

operations = {
    "add": operator.add,
    "subtract": operator.sub,
    "multiply": operator.mul,
    "divide": operator.truediv
}

action = "divide"
x, y = 10, 2
result = operations.get(action, lambda a, b: None)(x, y)
print(result)
#task05
```

Output:

A screenshot of a code editor interface. The top part shows a file named 'assginmet13.4.py' with a line number 10. The code in the editor is the same as the one in the previous block. The bottom part shows the output of the code, which is '5.0'. There is also a message 'Default timeout used 5.0' and 'Item Available'.

Observation:

The legacy code uses **multiple if-elif conditions** to choose the operation based on the action value.

This approach works but becomes **harder to maintain and scale** when more operations are added.

It can be refactored using a **dictionary mapping of actions to functions/operations**, making the code cleaner, more readable, and easier to extend while producing the same result 5.0

Task:05

Simplifying Search Logic in

Collections

Scenario

You are reviewing legacy inventory-check code that uses manual loops to find elements.

Task Description

- Identify the explicit loop used for searching an item.
- Use AI assistance to refactor the logic into a more concise and readable form.
- Maintain the same output behavior.

Legacy Code

```
inventory = ["pen", "notebook", "eraser", "marker"]
```

```
found = False
```

```
for item in inventory:
```

```
    if item == "eraser":
```

```
        found = True
```

```
    break
```

```
print("Item Available" if found else "Item Not Available")
```

Expected Output

Item Available

Prompt: Refactor the Python code that manually searches for an item in a list using a loop and a flag variable. Suggest a more concise and Pythonic approach while keeping the same output."

Code:

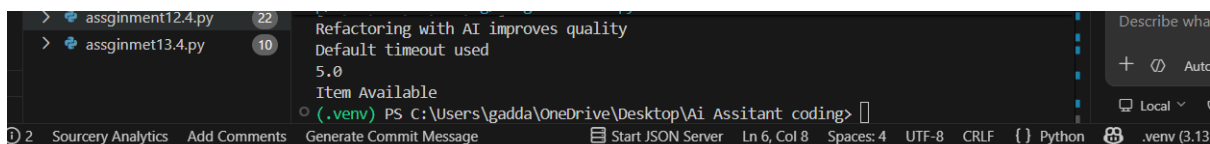


```
#(Variable) inventory: list[str]
inventory = ["pen", "notebook", "eraser", "marker"]

found = "eraser" in inventory

print("Item Available" if found else "Item Not Available")
```

Output:



```
> assginmet12.4.py 22
> assginmet13.4.py 10
Refactoring with AI improves quality
Default timeout used
5.0
Item Available
(.venv) PS C:\Users\gadda\OneDrive\Desktop\Ai Assitant coding>
```

Observation

The legacy code uses a **manual loop with a flag variable (found)** to search for an item in the list.

This approach works but is **longer and less Pythonic** for simple membership checks.

It can be simplified using the **in operator**, which directly checks if "eraser" exists in the list while keeping the same output.